Academic Paper Crawling Ecosystem —Research & Recommendations

> Compiled 2026-05-26 from GitHub search, API docs, and web research. Context: improving Academic Radar’ s `standardCrawler.js` / `trackerCrawl.js` pipeline.

Table of Contents

---

1. Top Open-Source Repos

Tier 1 —Directly Relevant (crawling + AI pipeline)

Repo	Stars	Language	What it does	Key takeaway for us
jannisborn/paperscraper	521	Python	Multi-source scraper: PubMed, arXiv, medRxiv, bioRxiv, chemRxiv. PDF download, citation count via Semantic Scholar, journal impact factors, plotting.	**Best architecture reference.** Downloads full arxiv dump locally for zero-API-cost keyword search. Fallback chain for PDF retrieval (BioC-PMC → eLife → Wiley/Elsevier). Uses SS_API_KEY env var.

Repo	Stars	Language	What it does	Key takeaway for us
TideDra/zotero-arxiv-daily	5.4k	Python	Recommends new arxiv papers based on Zotero library. Daily GitHub Actions run.	Interest-matching from user’s existing papers. Good recommendation model.
dw-dengwei/daily-arXiv-ai-enhanced	2.8k	JS	Daily arxiv crawl + AI summarization + GitHub Pages hosting.	**Direct ancestor of Academic Radar’s design.** HTML scraping discovery → API enrichment → LLM summary pipeline.
LearningCircuit/local-deep-research	8k	Python	Self-hosted deep research. 10+ search engines including arXiv, PubMed. Local & cloud LLMs. Retrieval with RAG.	Multi-engine architecture. Web search + academic sources combined.
kaixindelele/ChatPaper	19.5k	Python	ChatGPT-based arXiv paper summarization + translation + review.	Most popular. Chinese-first. Shows the demand for AI paper reading.
blazickjp/arxiv-mcp-server	2.8k	Python	MCP server for arXiv search/analysis.	Modern MCP-based approach —could be an integration pattern.

Tier 2 —Multi-Source Literature Review Tools

Repo	Stars	Language	Sources	Notes
jonatasgrosman/findpapers	358	Python	arXiv, Crossref, OpenAlex, Semantic Scholar, PubMed, Scopus, IEEE, Web of Science	Systematic literature review. Snowballing (forward/backward citation search).
monk1337/resp	483	Python	Google Scholar, arXiv, Semantic Scholar, ACL, PMLR, NeurIPS, OpenReview, CVF	Conference-specific scrapers. Citation graphs. Connected Papers integration.
LocalCitationNetwork/LocalCitationNetwork.github.io	138	JS	OpenAlex, Semantic Scholar, Crossref	Local citation network visualization for literature review.

Tier 3 —Specialized / Adjacent

Repo	Stars	What it does	Relevance
danielnsilva/semanticscholar	460	Unofficial Python client for Semantic Scholar APIs	Good wrapper pattern —we could adopt similar for Node.
Ar9av/PaperOrchestra	544	Automated paper writing with multi-agent	Shows multi-API integration (arXiv + S2).
jimmc414/onefilellm	2k	Scrapes arXiv, PubMed, URLs, repos into single LLM-prompt file	Useful for our text extraction pipeline.
Dianel555/paper-search-mcp-nodejs	162	Node.js MCP server for arXiv, PubMed, WoS, Semantic Scholar, Google Scholar	**Node.js based —closest to our stack.**

2. Academic APIs —Comparison Matrix

API	Key Required?	Rate Limit (no key)	Rate Limit (with key)	Coverage	PDF?	Citations?	Abstract?
Semantic Scholar	Free key recommended	Shared pool, throttled	1 RPS (intro tier)	214M papers, 79M authors, 2.49B citations	Yes (when available)	Yes (best citation data)	Yes + TLDRs
OpenAlex	No	~10 req/sec (polite pool)	Same	250M+ works, comprehensive	Links only	Yes (via referenced_works)	Yes (inverted index)
arXiv	No	1 req / 3 sec	Same	2.5M+ papers (CS, Math, Physics)	Yes (PDF direct)	No	Yes (API)
Crossref	No (email recommended)	~50 req/sec (polite pool with email)	Same	150M+ records, DOIs	No (links to publisher)	Yes (reference linking)	Only if provided by publisher
PubMed	No (API key optional)	3 req/sec	10 req/sec	37M+ biomedical	Some (PMC open access)	No	Yes
Unpaywall	No (email recommended)	100k/day (email), 1k/day (no email)	Same	50M+ OA articles	OA PDF links	No	No (metadata only)
GitHub	Token optional	60 req/hr	5000 req/hr	All public repos	N/A	Star/forks stats	Readme/description
Google Scholar	**No official API**	Very limited	Very limited	Unknown	No	Approximate count	Partial

Source Quality by Metric

```
Citation completeness:  Semantic Scholar > OpenAlex > Crossref > arXiv
PDF availability:       arXiv (direct) > Unpaywall (OA) > Semantic Scholar (links)
Abstract quality:       arXiv (author-written) > Semantic Scholar (TLDR) > OpenAlex (inverted)
Speed / reliability:    OpenAlex > Semantic Scholar > Crossref > arXiv > PubMed
Discovery breadth:      OpenAlex > Crossref > Semantic Scholar > arXiv > PubMed
Full text access:       arXiv (preprints) > PubMed Central (OA subset) > Unpaywall (OA links)
```

---

3. API Tokens You Should Provide

HIGH PRIORITY —Free, immediate improvement

Token / Config	Where to Get	Current Status	Impact
Semantic Scholar API Key	https://www.semanticscholar.org/product/api#api-key-form	☒ Not provided	Enables stable 1 RPS for citation-rich searches. Without it, you’re in the shared unauthenticated pool that gets throttled.
Crossref Email (`CROSSREF_EMAIL`)	Already in `.env` (empty)	⚠ Empty in .env	Moves you from “anonymous pool” to “polite pool” (better rate limits, faster responses). Just set to any email.
PubMed API Key + Email (`PUBMED_API_KEY`)	https://account.ncbi.nlm.nih.gov/settings/	☒ Not configured	Raises rate from 3/sec to 10/sec. Optional for non-biomedical use cases.

MEDIUM PRIORITY —Nice to have

Token / Config	Where to Get	Impact
GitHub Personal Access Token	https://github.com/settings/tokens	Raises from 60/hr to 5000/hr. Only needed if repo tracking is a core feature.

Token / Config	Where to Get	Impact
Semantic Scholar (higher tier)	Contact S2 team	Higher than 1 RPS. For production scale.

ALREADY CONFIGURED

Config	Status
`DEEPSEEK_API_KEY`	☒ Configured
`UNPAYWALL_EMAIL`	⚠ Empty —add email for 100× rate increase

How to Add These

```
# In your .env file:
SEMANTIC_SCHOLAR_API_KEY=your_key_here
CROSSREF_EMAIL=your_email@university.edu
PUBMED_API_KEY=your_ncbi_key
UNPAYWALL_EMAIL=your_email@university.edu
GITHUB_TOKEN=ghp_your_token
```

Then update `server/config.js` (the env vars are already defined there —just need values).

---

4. Architecture Patterns Worth Adopting

4.1 paperscraper's Local Dump Pattern (☒ Recommended)

Problem: arXiv API is slow (1 req/3 sec) and gets 429 rate limits.

Solution: Download the full arXiv metadata dump once (or periodically), store locally as JSONL. Then keyword search is instant and infinite.

```
Paperscraper approach:
  1. kaggle auth login           # one-time
  2. arxiv(start_date, end_date)  # downloads full dump
  3. search_keywords(query)       # instant, no API call
```

This is 1000× faster than calling the arXiv API per search. For a university with hundreds of teachers doing daily crawls, this is transformative. The dump is manageable (~2-3 GB for all of arXiv metadata).

Implementation for Academic Radar: Add a `paperSearch` provider that loads from a local JSONL dump instead of calling the API. Periodically refresh the dump.

4.2 findpapers' Snowballing Pattern

Problem: Keyword search misses related papers.

Solution: After initial keyword search, do forward/backward citation search: - Forward: papers that cite the found papers (find newer related work) - Backward: papers cited by the found papers (find foundational work)

This is trivial with Semantic Scholar's `/citations` and `/references` endpoints.

4.3 MCP Protocol Integration

Multiple repos (arxiv-mcp-server, paper-search-mcp-nodejs) now expose paper search as **Model Context Protocol** servers. This means LLMs can directly search for papers as a "tool call" during conversation. Your Academic Radar already has an MCP route —extending it to expose paper search would let the AI Center's LLM autonomously search when context is insufficient.

4.4 Multi-API Fallback Chain (paperscraper's PDF pattern)

```
Try: Direct PDF URL
  → Fallback: BioC-PMC (for biomedical OA papers)
  → Fallback: eLife GitHub (for eLife papers)
  → Fallback: Wiley TDM API (with API key)
  → Fallback: Elsevier TDM API (with API key)
  → Final: Return null (record "No PDF available")
```

Academic Radar could adopt a similar chain for PDF retrieval.

---

5. Priority Recommendations

Immediate (today)

1. **Get Semantic Scholar API key** —5 minutes, massive impact on citation/similarity data quality
2. **Set `CROSSREF_EMAIL`** —Literally any email address, immediate rate improvement
3. **Set `UNPAYWALL_EMAIL`** —Same as above

Short-term (this week)

4. **Add Semantic Scholar as a first-class source** in search ranking (it already exists as a provider in `paperSearch/providers/semanticScholar.js`, but give it higher priority in `ranking.js`)

5. **Implement retry timeout on arXiv** —Currently retries for 55+ seconds; cap at 30s total
6. **Make tracker generation async** —The `/api/trackers/generate` endpoint runs the full crawl synchronously (can take 60s+). Return tracker immediately, run crawl in background.

Medium-term

7. **Local arXiv dump** —Download full dump, enable instant keyword search
8. **Citation snowballing** —Forward/backward citation search via Semantic Scholar
9. **MCP paper search tool** —Expose `paperSearch` as an MCP tool for LLM autonomous search

Long-term / Production

10. **API key rotation / proxy pool** for high-volume crawls across multiple teachers
11. **Scheduled daily crawls** via the worker queue (already architecturally supported)

Appendix: Current Source Health in Academic Radar

From live testing on 2026-05-26:

Source	Status	Notes
OpenAlex	☒ Working	Fast, reliable. Primary source.
Semantic Scholar	☒ Provider exists	Needs API key for production use.
Crossref	⚠ Provider exists, untested	Needs email for polite pool.
arXiv	☒ Timeout/429	HTML scraping returns 400. API is rate-limited heavily. Retries take 55s+.
PubMed	⚠ Provider exists, untested	Only relevant for biomedical.
GitHub	☒ Working	For repositories. Needs token for scale.